

Simplified Newton–Raphson Power Flow: Reproduction, Complexity Audit and Comparative Evaluation*

CSE 402 Course Project — Group 12, Section B1

Student ID : 2105064, 2105067, 2105068, 2105081, 2105082

H M Aktaruzzaman Mukdho
Dept. of CSE, BUET
Dhaka, Bangladesh
2105064@cse.buet.ac.bd
aktaruzzamanmukdho@gmail.com

Ahtashamul Haque
Dept. of CSE, BUET
Dhaka, Bangladesh
2105067@cse.buet.ac.bd

Anika Morshed
Dept. of CSE, BUET
Dhaka, Bangladesh
2105068@cse.buet.ac.bd

Diganta Saha Tirtha
Dept. of CSE, BUET
Dhaka, Bangladesh
2105081@cse.buet.ac.bd

Jahedul Islam Nayeem
Dept. of CSE, BUET
Dhaka, Bangladesh
2105082@cse.buet.ac.bd

Abstract

Kulworawanichpong’s *simplified Newton–Raphson* power-flow method replaces the power-mismatch equations of the standard formulation with nodal current-mismatch equations. The roots are identical, but the Jacobian entries become far simpler expressions, so the paper claims a large reduction in per-iteration cost. This project reimplements both formulations from first principles, reproduces every reproducible table and figure of the base paper, and then subjects its central claim to an independent audit. Three findings emerge. First, the reproduction is exact where the paper is checkable: the 3-bus worked example matches the published Y-bus, Jacobian, mismatch vector and converged solution to every printed digit. Second, the paper’s complexity analysis is wrong in an identifiable way — its Table 1 counts off-diagonal work once per matrix *row* rather than once per *entry*, which makes the cost appear linear in the bus count. An element-by-element recount shows both methods are quadratic, separated by a constant factor of 1.67–1.80 rather than by a factor of n — the size of effect the paper’s own measurements show, where its Table 1 would have predicted $25.9\times$ at 57 buses. Its conclusion survives even though its arithmetic does not. Third, end-to-end the method is *slower*: lagging PV-bus reactive power destroys quadratic convergence and inflates the iteration count from 4 to 13 at 300 buses, and an augmented variant making reactive power a Newton unknown restores it. Finally, freezing and reusing the LU factorisation — the paper’s own premise carried further — yields a $2.85\times$ speed-up at 300 buses, where factorisation is 67% of an iteration and the Jacobian build the paper optimises only 30%.

Keywords

power flow, Newton–Raphson, current mismatch, cost model, sparse linear algebra, reproducibility

1 Introduction and Base Paper Review

1.1 Background

The power-flow (load-flow) problem asks for the steady-state complex voltage at every bus of an interconnected electrical network, given the real and reactive power injected or withdrawn at each bus. It is the computational kernel of essentially every power-system study — operational planning, contingency screening, stability assessment, electricity-market clearing — and is solved many thousands of times per day in control centres worldwide. Because the network equations are nonlinear, the problem is solved iteratively, and the cost of a single power-flow solution directly limits how many scenarios an operator can afford to examine.

Since Tinney and Hart’s formulation [2], the Newton–Raphson method has been the standard. It converges quadratically and typically reaches engineering accuracy in four to six iterations regardless of network size. Its cost per iteration, however, is dominated by two operations: rebuilding the Jacobian matrix from the current voltage estimate, and factorising it. A long line of research has attacked one or the other. The fast decoupled load flow of Stott and Alsac [3] replaces the Jacobian by two *constant* matrices that are factorised once, trading Newton’s convergence rate for a near-free iteration. Current-injection formulations in rectangular coordinates [4] make the bulk of the Jacobian constant while retaining Newton’s rate. The base paper of this project belongs to the same family: it leaves the Newton framework and the state variables untouched, and attacks only the arithmetic cost of forming the Jacobian entries.

1.2 The Base Paper

The selected base paper is T. Kulworawanichpong, “*Simplified Newton–Raphson power-flow solution method*”, International Journal of Electrical Power & Energy Systems **32** (2010) 551–558 [1].

Its proposal is compact. The standard method drives the *power* mismatch to zero,

$$\Delta S_k = S_{sch,k} - V_k \sum_i Y_{ki} \overline{V_i} = 0, \quad (1)$$

*Source code, test systems and the complete result set are available at https://github.com/Diganta81/numerical_project_402.

whereas the paper drives the nodal *current* mismatch to zero,

$$F_k = \left(\frac{S_{sch,k}}{V_k} \right) - \sum_{i=1}^n Y_{ki} V_i = 0. \quad (2)$$

Equations (1) and (2) have exactly the same roots — the second is the first divided by $\overline{V_k}$ — so nothing about the solution changes. What changes is the algebra of the derivatives. In the standard summand $|V_k V_i Y_{ki}| \cos(\theta_{ki} + \delta_i - \delta_k)$ both the near-end and the far-end bus appear; in the simplified summand $|Y_{ki} V_i| \cos(\theta_{ki} + \delta_i)$ there is no $|V_k|$ factor and no $-\delta_k$ in the angle. Every off-diagonal Jacobian entry therefore depends on a single bus index, and the diagonal entries become closed-form expressions instead of row sums.

The paper quantifies the saving in its Section 3 and claims that the Jacobian cost per iteration falls from $10n^2 + O(n)$ multiplications to $22(n-2)$ — that is, from quadratic to *linear* in the number of buses n . It supports this with a symbolic operation count (its Table 1) and two log-log plots (its Figures 1 and 2), and validates it experimentally on five test systems of 5 to 57 buses, reporting measured speed-ups of 1.13 to 1.73 (its Table 5 and Figure 5).

1.3 Objective and Contributions

The gap between a predicted speed-up of roughly $n/2$ and a measured speed-up of 1.13–1.73 is three orders of magnitude at the paper’s largest test case. The objective of this project is to determine, from a clean-room implementation, where that gap comes from — and whether the method is worth using. Concretely, this report contributes:

- (1) A from-scratch implementation of both formulations that reproduces the paper’s 3-bus worked example entry for entry (Section 4.1).
- (2) An independent audit of the paper’s complexity analysis, which locates a specific counting error in its Table 1 — visible in the paper’s own text — and replaces it with an element-by-element model whose predicted constant-factor saving is the size the paper actually measured, where its published model overestimates by more than an order of magnitude (Sections 4.2 and 4.3).
- (3) A stage-resolved timing decomposition that isolates the one step the paper’s analysis is actually about, and shows that it does speed up as predicted while the end-to-end solve does not (Sections 4.4 and 4.5).
- (4) Four extensions beyond the paper’s scope: scale testing to 300 buses, a comparison against four additional solvers, a memory study, and a robustness sweep to the voltage-stability boundary (Sections 4.6–4.9).
- (5) **PNR+**, an augmented variant of the paper’s method that treats PV-bus reactive power as a genuine Newton unknown. It removes the iteration-count penalty that makes the published method slower in practice.
- (6) A chord (frozen-Jacobian) study that carries the paper’s own premise to its conclusion and obtains a 2.85× end-to-end speed-up at 300 buses — the largest gain measured anywhere in this project (Section 4.10).

2 Methodology

2.1 Network Model and the Bus Admittance Matrix

A network of n buses and n_ℓ branches is described in per-unit on a common MVA base. Each bus carries a scheduled complex injection $S_{sch,k} = P_{sch,k} + jQ_{sch,k}$ (generation minus load) and is classified as slack (voltage magnitude and angle fixed), PV (magnitude and real power fixed) or PQ (both powers fixed). Each branch is a π -model with series impedance $r + jx$, total line charging b , an off-nominal tap ratio and a phase shift.

The bus admittance matrix is assembled branch by branch,

$$Y_{bus} = C^H Y_{br} C + \text{diag}(y_{sh}), \quad (3)$$

where C is the branch-bus incidence structure and y_{sh} collects the bus shunts. Y_{bus} is complex, symmetric when no phase shifters are present, and extremely sparse: its density falls from 0.76 on the 5-bus case to 0.012 on the 300-bus case. Both dense (NumPy) and sparse (SciPy CSR) assemblies are implemented and are verified against each other.

2.2 Standard Newton–Raphson (SNR)

The state vector is $x = [\delta_{pv \cup pq}; |V|_{pq}]$. The mismatch is the real and imaginary part of (1), restricted to the buses where each component is scheduled, and the Newton step solves

$$\begin{bmatrix} J_1 & J_2 \\ J_3 & J_4 \end{bmatrix} \begin{bmatrix} \Delta\delta \\ \Delta|V| \end{bmatrix} = \begin{bmatrix} \Delta P \\ \Delta Q \end{bmatrix}, \quad x \leftarrow x + \Delta x. \quad (4)$$

The sub-matrices are obtained from two complex derivative matrices $\partial S / \partial \delta$ and $\partial S / \partial |V|$, sliced into the reduced real system. This is the benchmark against which the paper measures itself, and it is implemented here in two independent ways — a vectorised form and a literal transcription of the textbook scalar equations — which are asserted equal in the test suite.

2.3 Simplified Newton–Raphson (PNR) — the Base Paper

The state vector is unchanged. Writing $V_k = |V_k| e^{j\delta_k}$, $Y_{ki} = |Y_{ki}| e^{j\theta_{ki}}$ and $S_{sch,k} = |S_{sch,k}| e^{j\phi_k}$ and splitting (2) into real and imaginary parts gives the paper’s equations (4) and (5),

$$G_k = \left| \frac{S_{sch,k}}{V_k} \right| \cos(\delta_k - \phi_k) - \sum_i |Y_{ki} V_i| \cos(\theta_{ki} + \delta_i) = 0, \quad (5)$$

$$H_k = \left| \frac{S_{sch,k}}{V_k} \right| \sin(\delta_k - \phi_k) - \sum_i |Y_{ki} V_i| \sin(\theta_{ki} + \delta_i) = 0. \quad (6)$$

Differentiating gives the paper’s equations (9)–(16). The off-diagonal entries depend on one bus index only, so an entire row of a sub-matrix shares the same trigonometric arguments, and the diagonals are closed-form rather than accumulated row sums. The paper writes its update as $[G; H] = J \Delta x$ followed by $x \leftarrow x + \Delta x$, so its J is $-\partial F / \partial x$; the implementation follows that sign convention exactly, which is why it reproduces the printed Jacobian matrices entry for entry.

2.4 PV Buses and the Augmented Variant (PNR+)

Equation (2) needs a *complete* complex $S_{sch,k}$, but at a PV bus only P is scheduled. This is the one place where the reformulation is not cost-free, and two treatments are implemented:

paper (default, faithful to the base paper) Q_k is re-evaluated from the *present* voltage estimate at the start of every iteration using the paper’s equation (22), and only the real current mismatch G_k is enforced. The bus contributes one equation and one unknown, exactly as ΔP_k does in the standard method.

augmented (PNR+) Q_k becomes a genuine Newton unknown and *both* G_k and H_k are enforced, so each PV bus contributes two equations and two unknowns. The extra Jacobian column is $-\partial F_k / \partial Q_k = j / \bar{V}_k$.

The distinction matters more than it appears. Lagging Q makes the PV buses a successive-substitution fixed point layered on top of Newton’s method, which costs the overall iteration its quadratic rate. Section 4.6 shows this is the single largest effect in the whole study.

2.5 Rectangular Current Injection (RCI)

Proposed as an extension, RCI writes the nodal current injection in rectangular components, $I_i = \sum_k (G_{ik} + jB_{ik})(e_k + jf_k)$, and takes the mismatch from its real and imaginary parts. The unknowns are (e_k, f_k) at every non-slack bus plus Q_k at every PV bus, with residuals

$$R_k^r = \frac{P_k e_k + Q_k f_k}{e_k^2 + f_k^2} - \sum_i (G_{ki} e_i - B_{ki} f_i), \quad (7)$$

$$R_k^m = \frac{P_k f_k - Q_k e_k}{e_k^2 + f_k^2} - \sum_i (B_{ki} e_i + G_{ki} f_i), \quad (8)$$

closed at PV buses by $R_k^V = |V_k^{sch}|^2 - (e_k^2 + f_k^2)$. The network part of the Jacobian is literally $\pm G$ and $\pm B$ taken straight from Y_{bus} and is therefore *constant*; only the 2×2 diagonal blocks and the PV columns change between iterations. The trade-off is the system dimension, $2(n-1) + n_{pv}$ against the polar methods’ $(n-1) + n_{pq}$.

2.6 Fast Decoupled Load Flow (FDLF)

FDLF exploits the weak coupling between P and $|V|$ and between Q and δ . The Jacobian is replaced by two constant, real, symmetric matrices built from the network susceptances,

$$B' \Delta \delta = \Delta P / |V|, \quad B'' \Delta |V| = \Delta Q / |V|, \quad (9)$$

factorised once before the iteration begins. Both standard variants are implemented: XB drops series resistance from B' , BX from B'' . FDLF is the natural control for this study, because it represents the limit of the paper’s own strategy — a Jacobian that costs nothing to rebuild — at the price of linear convergence.

2.7 Gauss–Seidel (GS)

Included as the classical baseline, GS applies the rearranged nodal equation bus by bus in place,

$$V_k \leftarrow \frac{1}{Y_{kk}} \left[\left(\frac{S_{sch,k}}{V_k} \right) - \sum_{i \neq k} Y_{ki} V_i \right], \quad (10)$$

over-relaxed by an acceleration factor $\alpha = 1.6$. At a PV bus the reactive injection is estimated from the present voltages and the updated voltage is rescaled onto its magnitude set-point.

2.8 Chord (Frozen-Jacobian) Variants

A further extension implements *chord* iterations of both NR formulations: the Jacobian is built and LU-factorised once (or refreshed every k iterations), and subsequent steps reuse the stored factors through forward and back substitution alone. This isolates the question the base paper raises — how much of an iteration is really the Jacobian rebuild — by removing that cost entirely rather than reducing it. Section 4.10 reports the outcome, which turns out to be the largest single result of the project.

2.9 Operation-Counting Models

Three independent models of per-iteration arithmetic cost are implemented. Following the paper’s own convention, a “FLOP” is a multiplication or a division; additions, subtractions and transcendental evaluations are treated as free.

paper The counts exactly as published, reproducing Table 1 and Figures 1–2 of the paper.

audited An element-by-element recount of the *same* formulas, granting both methods identical common-subexpression reuse.

sparse The audited count with the dense off-diagonal population $(n-1)(n-2)$ replaced by the true number of non-zeros of Y_{bus} .

With d off-diagonal entries per row, the audited unit costs of Table 7 give a predicted Jacobian cost ratio

$$r(d) = \frac{9d + 4}{5d + 13}, \quad (11)$$

whose break-even point $r(d) = 1$ and dense asymptote $9/5$ are central to Section 4.3.

3 Implementation and Experiments

3.1 Software Architecture

The implementation is written from scratch in Python with NumPy and SciPy, and is organised as a library (`src/powerflow/`) with no I/O side effects, driven by eight standalone experiment scripts. The library separates four concerns:

- `case.py`, `loader.py` — a solver-agnostic PowerCase data model and a reader for MATPOWER-format JSON [8]. Every solver consumes this one object, which is what keeps the comparison honest: the methods differ only in the equations they iterate on, never in the data they see.
- `ybus.py`, `results.py` — admittance-matrix construction (dense and sparse), branch flows, and a `PowerFlowResult` carrying the full per-iteration history.

Table 1: Solvers implemented and compared.

Key	Formulation
SNR	Standard NR, power mismatch, polar state
PNR	Simplified NR, current mismatch (base paper)
PNR+	As PNR, PV reactive power a Newton unknown
RCI	Rectangular current injection
FDLF-XB	Fast decoupled, resistance dropped from B'
FDLF-BX	Fast decoupled, resistance dropped from B''
GS	Gauss-Seidel, $\alpha = 1.6$

Table 2: Test systems. d is the average number of off-diagonal non-zeros per row of Y_{bus} , the quantity that governs equation (11).

Label	System	n	n_ℓ	n_{pv}	n_{pq}	d
-	case3_saadat	3	3	1	1	2.00
TC1	case5_stagg	5	7	0	4	2.80
TC2	case6ww	6	11	2	3	3.67
-	case14_ieee	14	20	4	9	2.86
TC3	case24_ieee_rts	24	38	10	13	2.83
TC4	case30_ieee	30	41	5	24	2.73
TC5	case57_ieee	57	80	6	50	2.74
EX1	case118_ieee	118	186	53	64	3.03
EX2	case300_ieee	300	411	68	231	2.73

- solvers/ — seven interchangeable solvers behind one entry point, solve(case, options, ybus).
- flops.py, benchmark.py, plotting.py, reporting.py — the cost models, the timing and memory harness, and figure/table output.

The seven solver keys used throughout this report are summarised in Table 1.

3.2 Test Systems

Eight networks are used, listed in Table 2. They are the classical textbook systems of Stagg and El-Abiad [5] and Saadat [6] together with the IEEE test cases distributed by the University of Washington archive [9] in MATPOWER format [8]. The 3-bus system is the paper’s own worked example, back-derived from the Y-bus it prints. TC1–TC5 correspond to the five test systems of the paper’s Section 5. EX1 and EX2 are added by this project to probe the regime the paper never reaches.

A caveat on the paper’s Table 3. The paper states that its test systems were “modified” but never says how, and the bus voltages printed in its Table 3 are not those of the standard IEEE cases. Several of its 24-bus voltages sit near $0.46 - j0.91$ p.u.: the magnitude is normal, but the angle is -63° from the slack bus, a spread no normally-operating transmission network shows. The unmodified library cases are therefore used here, and the paper’s Table 3 is *not* reproducible number for number; the solutions this project obtains for TC1–TC5 are tabulated in 03_table3_solutions, where the two NR formulations agree with each other to better than 10^{-9} p.u.

on every bus. Iteration counts, convergence shapes and timing ratios *are* reproducible, and they are what the paper’s claims actually rest on.

3.3 Experimental Protocol

Unless stated otherwise, every run uses:

- a **flat start** ($|V| = 1$ p.u. at PQ buses, set-point elsewhere, all angles zero);
- the paper’s own termination test — a *maximum voltage correction* of 10^{-6} p.u., not a mismatch tolerance;
- dense linear algebra, except where the dense/sparse comparison is the subject;
- timing as the **best of** 10–25 repeats, with Y_{bus} construction excluded, so that the figure reported is the one least contaminated by scheduler noise.

One exception applies to the chord experiment of Section 4.10, which pins BLAS and LAPACK to a single thread: on matrices of a few hundred rows a multi-threaded BLAS spends more time waking threads than computing, which is unfair to iterations consisting only of triangular solves. Its absolute timings are therefore not comparable with the other tables, though the ratios within it are.

Memory is measured as the peak heap increment over one solve using Python’s tracing allocator, which counts only allocations the solver itself makes. All measurements were taken on an AMD Ryzen 7 5800H (16 threads) under Python 3.14.7, NumPy 2.5.3, SciPy 1.18.1, pandas 3.0.6 and matplotlib 3.11.2 [10–13].

3.4 Verification Strategy

Correctness is established before any performance claim is made. The test suite contains 285 tests covering four levels:

- (1) **Against the paper.** The Y-bus, the mismatch vector, the full Jacobian and the correction vector of all three printed iterations of the worked example are compared entry by entry with the published values.
- (2) **Internal consistency.** The vectorised Jacobians are asserted equal to literal transcriptions of the scalar textbook equations, and sparse assemblies equal to dense ones.
- (3) **Cross-method equivalence.** Every converged solver must zero the *power* mismatch regardless of the residual it iterated on, respect the slack and PV set-points, and agree with a standard-NR reference driven to 10^{-13} p.u.
- (4) **Behavioural.** Warm starts, iteration limits, PV $\rightarrow$ PQ switching under generator reactive limits, and monotonicity of the error history.

The full suite passes.

Data availability. Every figure in this report has a companion machine-readable table of the same name in results/tables/, written as CSV and Markdown. Five result tables are too long to typeset and are referenced in place instead: the bus-by-bus solutions of all five test systems (03_table3_solutions, 122 rows), the per-iteration convergence histories behind Figures 6–10 of the paper (03_convergence_histories) and behind the method comparison (05_convergence_30, 05_convergence_118), and the full loading sweep behind the robustness heat maps (07_robustness, 300 rows). The chord experiment writes to results/extension_chord/, whose

Table 3: Computed versus printed Y_{bus} (magnitude in p.u., angle in degrees).

Entry	Comp. mag.	Paper	Comp. ang.	Paper	Abs. err.
Y_{11}	53.85165	53.8517	-68.19859	-68.20	$5.19\text{e-}05$
Y_{12}	22.36068	22.3607	116.56505	116.57	$2.02\text{e-}05$
Y_{13}	31.62278	31.6228	108.43495	108.43	$2.34\text{e-}05$
Y_{22}	58.13777	58.1378	-63.43495	-63.43	$3.26\text{e-}05$
Y_{23}	35.77709	35.7771	116.56505	116.57	$1.24\text{e-}05$
Y_{33}	67.23095	67.2310	-67.24902	-67.25	$5.47\text{e-}05$

Table 4: Per-iteration agreement with the values printed in Section 4 of the paper.

Iter.	Quantity	Max. abs. diff.	Max. rel. diff.
1	mismatch	$2.31\text{e-}05$	$8.07\text{e-}06$
1	Jacobian	0	0
1	Δx	$3.66\text{e-}05$	$7.95\text{e-}04$
2	mismatch	$4.34\text{e-}05$	$5.25\text{e-}04$
2	Jacobian	$4.20\text{e-}05$	$6.65\text{e-}07$
2	Δx	$4.25\text{e-}05$	$3.86\text{e-}02$
3	mismatch	$4.31\text{e-}07$	$1.07\text{e-}03$
3	Jacobian	$3.89\text{e-}05$	$6.15\text{e-}07$
3	Δx	$3.75\text{e-}08$	$3.68\text{e-}03$

per-iteration traces (`08_convergence_histories`) are likewise referenced rather than typeset.

4 Results and Discussion

4.1 Exact Reproduction of the Worked Example

The paper prints, for its 3-bus example, the full Y_{bus} , and then the current mismatch vector, the Jacobian and the correction vector for three iterations. All of it reproduces.

Table 3 compares the computed admittance matrix against the printed one. Agreement is to the four decimal places the paper itself reports; the residual error, at most 5.5×10^{-5} in magnitude, is consistent with the paper’s own rounding rather than with a modelling difference.

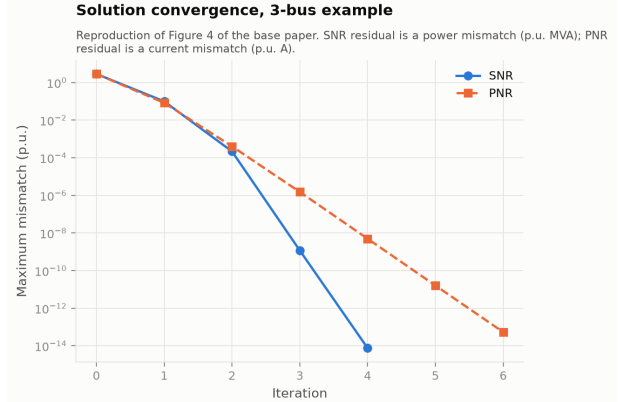
Table 4 carries the per-iteration comparison. The first Jacobian matches to *every printed digit* — an exact zero difference — and the later iterations differ only at the level the paper’s four-decimal rounding of its own intermediate voltages propagates to. The relative column is normalised by the largest entry of the paper’s own vector or matrix and therefore absorbs that rounding.

The converged solution (Table 5) matches the paper’s Table 2 on every quantity. The small residual gap in the angle columns arises because the paper quotes its third-iteration values while this work iterates to 10^{-10} p.u.; the standard method, which the paper also reports, agrees to all four printed digits.

Figure 1 reproduces the paper’s Figure 4. Over the three iterations the paper plots, the two methods track each other closely and end at 2.17×10^{-4} and 4.03×10^{-4} — the values its Table 2 records as 2.2×10^{-4} p.u. power and 4.0×10^{-4} p.u. current. The paper’s own conclusion for this example, that there is no significant difference between the two convergences, is reproduced exactly.

Table 5: Paper Table 2 — converged solution of the 3-bus example.

Item	SNR (ours)	SNR (paper)	PNR (ours)	PNR (paper)
$ V_2 $ (p.u.)	0.971680	0.97168	0.971680	0.97168
δ_2 (deg)	-2.6965	-2.696	-2.6965	-2.698
$ V_3 $ (p.u.)	1.040000	1.04000	1.040000	1.04000
δ_3 (deg)	-0.4988	-0.4988	-0.4988	-0.4979
Q_3 (p.u.)	1.461769	1.46170	1.461769	1.46180
Max. mismatch	$2.17\text{e-}4$	$2.2\text{e-}4$	$4.03\text{e-}4$	$4.0\text{e-}4$

**Figure 1: Reproduction of the paper’s Figure 4: maximum mismatch per iteration for the 3-bus example. The SNR residual is a power mismatch (p.u. MVA); the PNR residual is a current mismatch (p.u. A).**

Carried past the window the paper plots, they separate. Both show the roughly-doubling number of correct digits per iteration characteristic of quadratic Newton convergence, but the standard method reaches machine precision in four iterations (7.6×10^{-15}) while the simplified method needs six (5.4×10^{-14}). On a system with a single PV bus the gap is one the paper had no reason to notice; it is nonetheless the first visible symptom of the PV-bus treatment of Section 2.4, and Section 4.6 shows what it becomes at scale.

4.2 Operation Counts: Published versus Audited

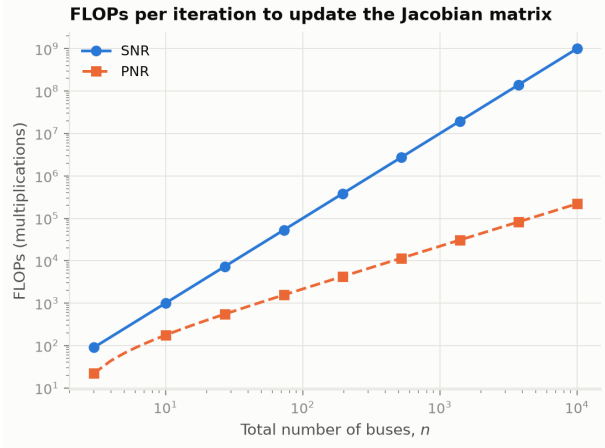
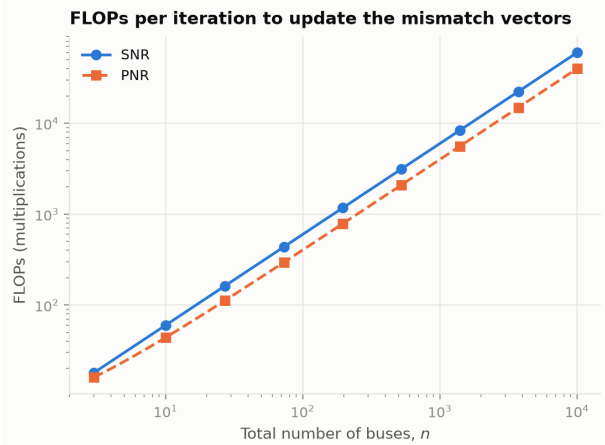
The paper’s symbolic counts reproduce exactly. Every form is recovered - $3(n-1)$ and $3(n-1)(n-2)$ for the diagonal and off-diagonal blocks of J_1 and J_3 under SNR, $2(n-1) + 3$ and $2(n-1)(n-2)$ for J_2 and J_4 , against $4(n-2)$ and $2(n-2)$ or $(n-2)$ for PNR. Table 6 gives the resulting totals, and Figures 2 and 3 reproduce the paper’s Figures 1 and 2.

The counts cannot be right as stated. A dense Jacobian has $O(n^2)$ entries, and the paper’s own equations (9), (11), (13) and (15) give every one of them a *distinct* admittance Y_{ki} . Each entry therefore needs at least one multiplication of its own, and a cost linear in n is impossible.

The slip is visible in the paper’s own derivation, and does not have to be inferred. Discussing J_1 , it states that equation (9) costs

Table 6: Paper Table 1 — multiplications per iteration to build the Jacobian, as published.

n	SNR total	PNR total	Claimed ratio
5	166	66	2.5
6	256	88	2.9
24	5 296	484	10.9
30	8 416	616	13.7
57	31 366	1 210	25.9
118	136 896	2 552	53.6
300	894 016	6 556	136.4

**Figure 2: Paper Figure 1 reproduced: Jacobian multiplications per iteration, $10n^2$ (SNR) against $22(n-2)$ (PNR).****Figure 3: Paper Figure 2 reproduced: mismatch-vector multiplications per iteration, $6n$ (SNR) against $4n+4$ (PNR).**

two multiplications per element and equation (19), its standard-method counterpart, three; that there are $(n-1) \times (n-2)$ off-diagonal elements; and therefore that the standard method needs $3 \times (n-1) \times (n-2)$. It then charges the proposed method $2 \times (n-2)$

Table 7: Audited multiplication counts per matrix element.

Method	Off-diag. pair	Bus (diag.)	Mismatch term
Standard NR (SNR)	9	4	4
Simplified NR (PNR)	5	13	3

Table 8: Per-iteration cost ratio SNR/PNR: as published versus audited.

n	Audited SNR	Audited PNR	Audited	Published
5	192	163	1.18	3.1
6	304	243	1.25	3.4
24	6 766	4 419	1.53	10.1
30	10 792	6 963	1.55	12.4
57	40 492	25 539	1.59	22.8
118	177 376	110 451	1.61	46.2
300	1 160 722	717 603	1.62	116.2
1000	12 969 022	7 992 003	1.62	385.4

— two multiplications applied to one *row*’s worth of elements, not to the $(n-1) \times (n-2)$ it had just counted. The same substitution is made for J_2 , J_3 and J_4 , and it is exactly what turns Table 1’s bottom line from quadratic into linear.

Applying the paper’s own per-element costs consistently already removes the effect: summing its four sub-matrices gives $3+2+3+2 = 10$ multiplications per off-diagonal entry for the standard method against $2+1+2+1 = 6$ for the proposed one, a ratio of $10/6 = 1.67$ — a constant, with both methods $O(n^2)$.

An honest recount. To test whether the paper’s *conclusion* survives its arithmetic, the same formulas were recounted element by element with identical common-subexpression reuse granted to both methods. For an off-diagonal pair (k, i) the standard method must form $u = |V_i||Y_{ki}|$, $w = |V_k||Y_{ki}|$ and $m = |V_k|u$ and then multiply by $\sin \alpha$ and $\cos \alpha$ with $\alpha = \theta_{ki} + \delta_i - \delta_k$, filling all four sub-matrices and their row sums: nine multiplications. The simplified summand carries neither the $|V_k|$ factor nor $-\delta_k$, so only u is needed and the count falls to five. Against that, the simplified diagonals are closed-form expressions rather than row sums the standard method accumulates for free, and cost thirteen multiplications per bus against four. Table 7 collects the unit costs.

The consequence is a constant-factor saving, not an asymptotic one (Table 8, Figure 4). Both methods remain $O(n^2)$; the ratio tends to $9/5 = 1.80$ for the Jacobian alone, and including the mismatch vectors — which save only 4 : 3 — brings the combined figure to about 1.62.

The conclusion survives. The two models differ by more than an order of magnitude in what they predict, and only one of them is the size of anything the paper measured. At 57 buses, its largest test case, Table 1 predicts a Jacobian saving of 25.9 $\times$; the corrected models predict 1.67–1.80 $\times$; and the paper’s own Table 5 reports total-time ratios of 1.13–1.73 $\times$ across all five systems. The corrected count is the right order of magnitude throughout, the published one is not.

The comparison should not be pushed further than that. The paper’s 1.728 is a *total solve time* ratio that also embeds a 4-versus-3

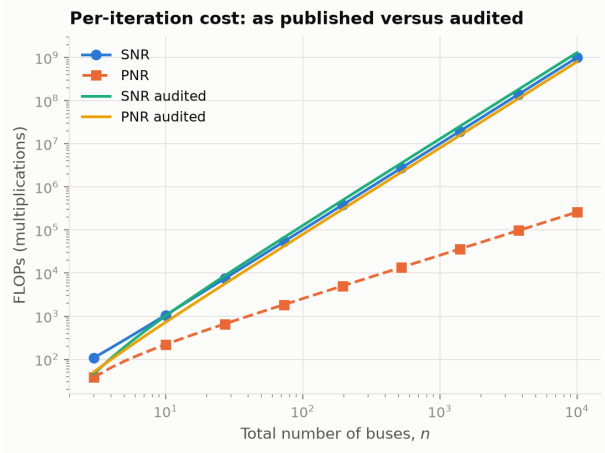


Figure 4: Per-iteration cost as published against the audited recount of the same formulas. The published PNR curve is linear in n ; the audited one is quadratic and parallel to SNR, separated by a constant factor.

Table 9: Audited FLOP counts over the true Y_{bus} sparsity pattern.

Case	n	nnz	d	Sp. SNR	Sp. PNR	Ratio
TC1	5	19	2.80	218	179	1.22
TC2	6	28	3.67	330	259	1.27
TC3	24	92	2.83	1 072	915	1.17
TC4	30	112	2.73	1 302	1 123	1.16
TC5	57	213	2.74	2 480	2 147	1.16
EX1	118	476	3.03	5 594	4 739	1.18
EX2	300	1 118	2.73	13 030	11 331	1.15

iteration advantage on that case, so it is not a per-iteration Jacobian measurement and cannot be set against 1.73 digit for digit. What the agreement in magnitude does establish is the central analytical result of this project: the base paper reaches a correct conclusion through an incorrect calculation, and the corrected calculation is consistent with its experimental data where the published one is off by a factor of fifteen. Section 4.5 measures the per-iteration quantity directly.

4.3 Cost over the True Sparsity Pattern

The counts above, like the paper’s, assume a dense Jacobian. Real networks are not dense. Replacing the dense off-diagonal population with the true non-zero count of Y_{bus} gives Table 9.

Two things change once sparsity is admitted. First, the quadratic term never materialises: at 300 buses a dense implementation performs 89× more multiplications than a sparse one for the same result, so the $O(n^2)$ versus $O(n)$ argument the paper builds its case on concerns an implementation nobody would write. Second, and more damaging, the advantage of the simplified method nearly disappears — the ratio column sits at 1.15–1.27 rather than 1.80.

The reason follows directly from equation (11). Break-even sits at $d = 2.25$: a network sparser than that gives the simplified Jacobian

Table 10: Predicted Jacobian cost ratio versus network density.

Case	n	d	$r(d)$ sparse	$r(n-1)$ dense
TC1	5	2.80	1.081	1.212
TC2	6	3.67	1.181	1.289
TC3	24	2.83	1.086	1.648
TC4	30	2.73	1.073	1.677
TC5	57	2.74	1.073	1.734
EX1	118	3.03	1.111	1.768
EX2	300	2.73	1.072	1.787

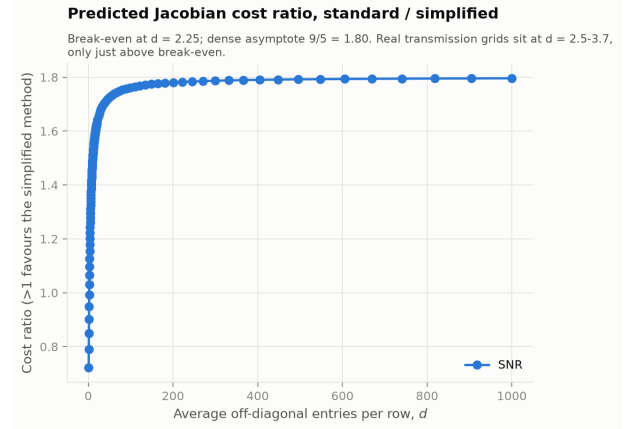


Figure 5: Predicted Jacobian cost ratio $r(d) = (9d + 4)/(5d + 13)$. Break-even at $d = 2.25$; dense asymptote $9/5 = 1.80$. Real transmission grids sit at $d \approx 2.5$ – 3.7 , only just above break-even.

no advantage at all, because its closed-form diagonals cost more than the row sums they replace. The transmission systems studied here sit at $d = 2.73$ – 3.67 , only just above break-even (Table 10). Figure 5 plots the relation across the whole range and Figure 6 plots the sparse counts against system size.

The prediction is thus sharper than the paper’s: the simplified method’s advantage is real but modest, it is largest on a dense formulation such as the paper’s, and it shrinks to a few per cent once a production-grade sparse solver is used. The next sections test that prediction against the clock.

4.4 The Paper’s Table 5: Iterations and Solve Time

Table 11 is the direct reproduction of the paper’s Table 5 on the five test systems, with the paper’s own figures alongside. It is the point at which this study and the base paper part company.

Three observations follow.

Iteration counts do not match, and the direction is systematic. The paper reports PNR needing *fewer or equal* iterations on four of five cases. This implementation finds the opposite on four of five: PNR needs the same number only on TC1 (which has no PV buses at all) and more everywhere else, with the largest gap on TC3, the most

Table 11: Reproduction of the paper’s Table 5. time ratio is total solve time SNR/PNR — values above 1 mean the simplified method was faster. per-iter. ratio divides out the differing iteration counts and is the quantity the paper’s FLOP analysis actually predicts. Best of 25 repeats, Y_{bus} construction excluded.

Case	System	n	Iterations (ours)		Iterations (paper)		Time (ms, ours)		Ratios		
			SNR	PNR	SNR	PNR	SNR	PNR	time (ours)	time (paper)	per-iter. (ours)
TC1	case5_stagg	5	4	4	5	4	0.503	0.507	0.990	1.539	0.990
TC2	case6ww	6	4	5	5	5	0.494	0.651	0.758	1.459	0.948
TC3	case24_ieee_rts	24	4	7	7	8	0.621	1.057	0.588	1.132	1.028
TC4	case30_ieee	30	4	5	5	5	0.671	0.860	0.780	1.329	0.975
TC5	case57_ieee	57	5	6	4	3	1.839	2.202	0.836	1.728	1.003

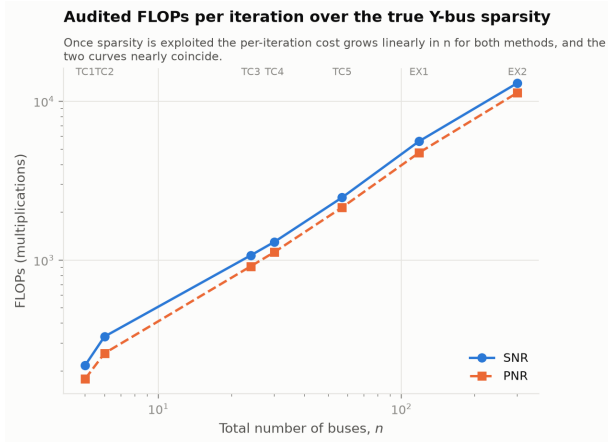


Figure 6: Audited FLOPs per iteration over the true Y_{bus} sparsity. Once sparsity is exploited the cost grows linearly in n for both methods and the two curves nearly coincide.

PV-heavy case ($n_{pv} = 10$ of 24 buses). That pattern — the penalty scaling with the PV-bus count — is the signature of the lagged- Q treatment of Section 2.4, and it recurs in every later experiment.

End-to-end, the simplified method is slower. Every measured time ratio is below 1 (mean 0.79), against the paper’s 1.13–1.73. Figures 7 and 8 show the measured times and set the two sets of ratios side by side.

Per iteration, the two methods are indistinguishable. Dividing out the iteration counts gives ratios of 0.95–1.03 — unity to within measurement noise. This is the crucial number: it says the FLOP-level saving established in Section 4.2 is *not visible* in the wall-clock cost of a whole iteration. Section 4.5 explains why.

Figure 9 shows the iteration counts directly, and Figure 10 reproduces the paper’s Figures 6–10, the convergence histories of each test case. Their *shape* is as informative as their length: the SNR curves steepen (quadratic convergence), while the PNR curves on the PV-heavy cases straighten into the constant slope of linear convergence. The underlying per-iteration data is tabulated in 03_convergence_histories.

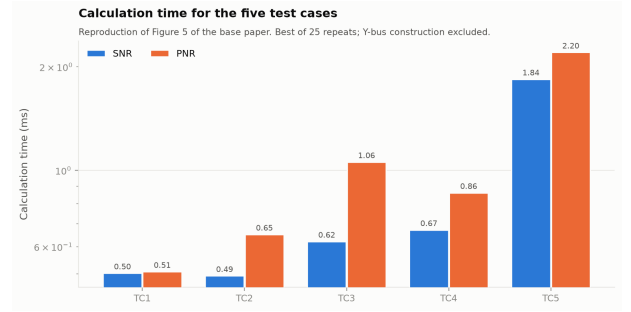


Figure 7: Reproduction of the paper’s Figure 5: calculation time for the five test cases.

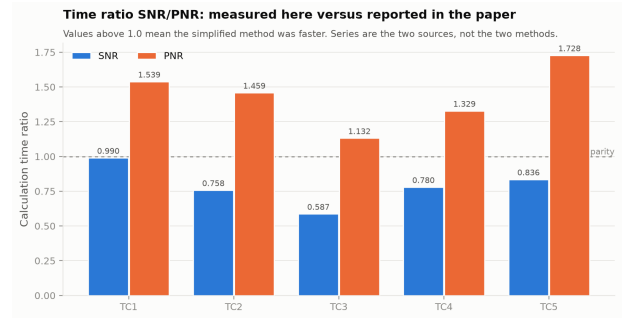


Figure 8: Measured time ratio SNR/PNR against the ratio reported in the paper. Values above 1.0 mean the simplified method was faster.

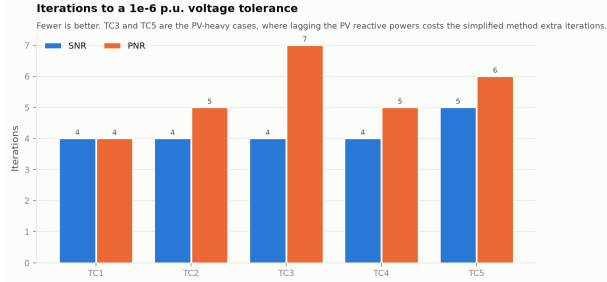
4.5 Where the Saving Actually Lives

To reconcile a genuine arithmetic saving with an invisible wall-clock effect, one Newton iteration was instrumented and split into four stages: evaluating the mismatch vector, evaluating the Jacobian derivatives, assembling the reduced real block matrix, and the LU solve. Only the first two differ between the methods; the last two are *identical work*. Table 12 reports the decomposition.

The decomposition settles the question. At the one step the paper’s analysis is about — evaluating equations (9)–(16) instead of the standard derivative formulas — the simplified method is faster, by 1.21 to 1.31 $\times$, in the same range as the audited prediction of Section 4.3 (1.15–1.27 at these densities). But that step is only a

Table 12: One Newton iteration decomposed. deriv. ratio isolates the paper’s claim; shared share is the fraction of an iteration spent on work that is identical for both methods and therefore cannot be improved by reformulating the mismatch.

System	n	Deriv. SNR (μ s)	Deriv. PNR (μ s)	Deriv. ratio	Mismatch SNR	Mismatch PNR	Assembly	LU solve	Shared share
case5_stagg	5	13.81	11.41	1.21	12.08	11.84	22.02	4.83	0.509
case6ww	6	14.34	11.40	1.26	12.76	11.94	23.10	4.99	0.509
case24_ieee_rts	24	18.84	14.70	1.28	12.92	12.39	28.79	11.02	0.556
case30_ieee	30	21.05	16.03	1.31	13.45	12.78	33.75	18.24	0.601
case57_ieee	57	34.70	35.65	0.97	13.89	20.43	60.69	78.89	0.742

**Figure 9: Iterations to a 10^{-6} p.u. voltage tolerance on TC1–TC5. Fewer is better. TC3 and TC5 are the PV-heavy cases.**

quarter of an iteration. Between 51% and 74% of every iteration is block assembly plus the LU solve, which are byte-for-byte identical work in the two formulations. By Amdahl’s argument a $1.3\times$ speed-up on a 27% fraction yields about 6% overall — which is exactly the per-iteration ratio of 0.95–1.03 observed in Table 11, lost in the noise. Figure 11 plots the derivative stage on its own.

The 57-bus row is an outlier (ratio 0.97) and is reported as measured; the neighbouring sizes bracket it at 1.31 (30 buses) and 1.51 (118 buses, see Table 15), so it is attributed to a measurement artefact at the matrix size where the BLAS switches strategy rather than to a property of the formulations.

4.6 Extension 1: Scaling to 300 Buses

The paper stops at 57 buses. Table 13 sweeps the four Newton formulations from 3 to 300.

This is the project’s clearest result. The standard method’s iteration count is essentially flat across two orders of magnitude of system size — 4 to 5 iterations from 3 buses to 300 — which is the textbook behaviour of a quadratically convergent method. The paper’s method starts level with it and then degrades steadily, reaching **13 iterations at 300 buses against SNR’s 5**. Fitting a power law to iteration count against bus count (Table 14) makes the separation quantitative: SNR’s exponent is 0.041, statistically indistinguishable from the zero growth Newton’s method promises, while PNR’s is 0.251, six times larger.

The per-iteration exponents tell the complementary story: all four methods sit between 1.37 and 1.46, confirming that per-iteration cost is governed by the shared assembly and factorisation stages (between $O(n^2)$ assembly and $O(n^3)$ dense LU), not by the formulation of the derivatives.

PNR+ repairs the defect. Making reactive power a genuine Newton unknown restores an almost-flat iteration count — exponent 0.101, within one iteration of SNR on every system tested — while retaining the cheaper derivative evaluation. At 300 buses PNR+ solves in 30.4 ms against PNR’s 53.9 ms, a $1.77\times$ improvement obtained purely by fixing the PV treatment. RCI behaves similarly (6 iterations at 300 buses), confirming that the deficiency is specific to lagging Q and not to current-mismatch formulations in general. Figures 12 and 13 show both trends.

The derivative advantage does grow, as predicted. Table 15 and Figure 14 isolate the derivative stage across the full size range. The ratio climbs from 1.13 at 3 buses to 1.60 at 300 — approaching the audited dense asymptote of 1.80 — while the whole-iteration ratio never leaves the neighbourhood of 1.1. The paper’s mechanism is real; it is simply confined to a minority of the work.

Sparse storage is the larger lever, and it is orthogonal. Table 16 compares dense and sparse linear algebra on the same solver. Sparse storage is a loss at 57 and 118 buses — SciPy’s sparse LU carries enough constant overhead to be $2\text{--}4\times$ slower than LAPACK on matrices of a few hundred rows — and becomes a $2\times$ gain at 300 buses, with the gap widening from there. Crucially, the benefit is identical for both formulations, so it is orthogonal to the base paper’s contribution, and at large n it is far larger than anything the reformulation offers.

4.7 Extension 2: Comparison Against Four More Solvers

Table 17 places all seven solvers on all seven systems.

The ordering spans the full trade-off the base paper operates within. Newton methods need 4–13 iterations; FDLF needs 6–11; Gauss–Seidel needs 25–378 and *fails outright* on the 300-bus system, exhausting a 3000-sweep budget at a residual of 1.6×10^{-3} p.u. Per iteration the ordering reverses: GS is cheapest, then FDLF, then the Newton family.

Two results deserve emphasis. First, FDLF-BX is the fastest solver on the 118-bus system (2.35 ms, beating SNR’s 2.72 ms) despite needing half again as many iterations — because it never rebuilds a Jacobian at all. That is the limiting case of the base paper’s own strategy, and it shows the strategy is sound even though the paper’s particular realisation of it is not aggressive enough to pay off. Second, at 300 buses the standard method is fastest overall and PNR is $2.5\times$ slower, which is the opposite of the paper’s conclusion extrapolated to that size.

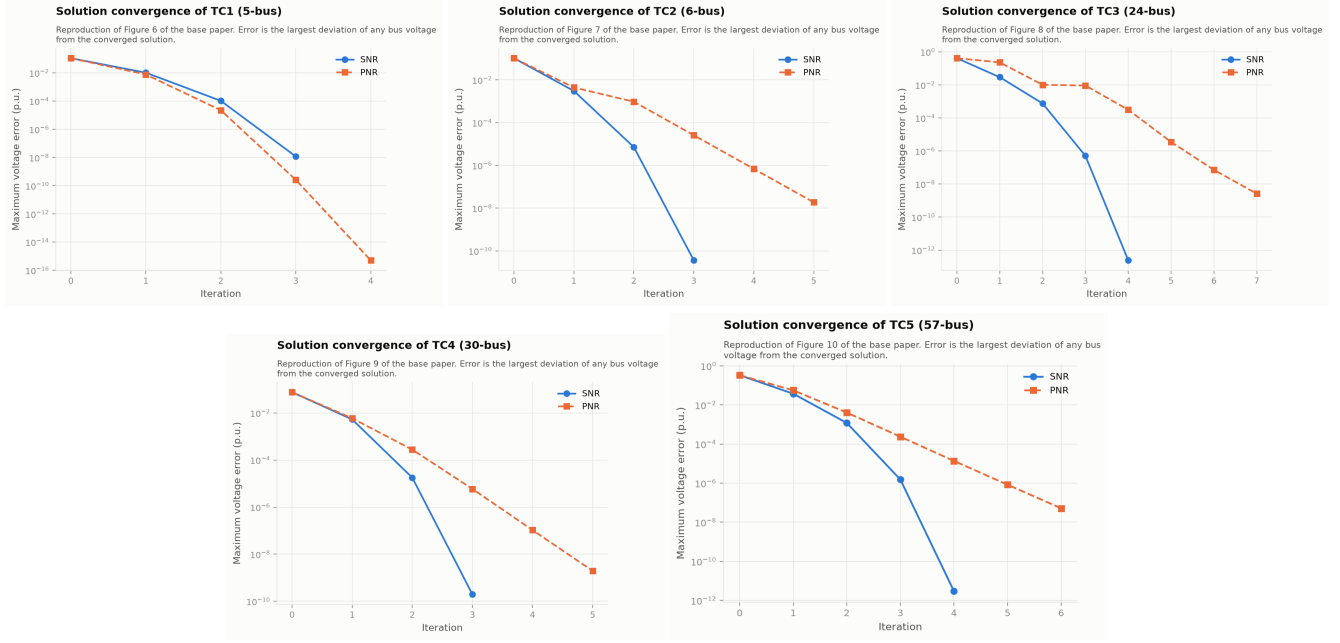


Figure 10: Reproduction of the paper’s Figures 6–10: maximum voltage error per iteration. Top row, left to right: TC1 (5-bus), TC2 (6-bus), TC3 (24-bus). Bottom row: TC4 (30-bus), TC5 (57-bus). A curve that steepens is quadratic convergence; a straight line is linear.

Table 13: Solver sweep from 3 to 300 buses. Flat start, 10^{-6} p.u. voltage tolerance, best of 15 repeats, dense linear algebra. rows is the dimension of the linear system factorised each iteration.

System	n	Iterations				Solve time (ms)				Rows		
		SNR	PNR	PNR+	RCI	SNR	PNR	PNR+	RCI	SNR/PNR	PNR+	RCI
case3_saadat	3	4	4	3	3	0.48	0.52	0.40	0.52	3	4	5
case5_stagg	5	4	4	4	3	0.49	0.50	0.52	0.49	8	8	8
case6ww	6	4	5	4	4	0.52	0.71	0.58	0.72	8	10	12
case14_ieee	14	4	7	5	5	0.55	0.96	0.70	0.90	22	26	30
case24_ieee_rts	24	4	7	5	5	0.58	1.01	0.78	1.01	36	46	56
case30_ieee	30	4	5	4	4	0.64	0.82	0.69	0.87	53	58	63
case57_ieee	57	5	6	4	4	1.65	2.20	1.57	1.86	106	112	118
case118_ieee	118	4	12	5	6	2.85	8.23	4.85	6.94	181	234	287
case300_ieee	300	5	13	6	6	21.40	53.90	30.42	31.48	530	598	666

Table 14: Fitted growth exponents (log–log slope of cost against bus count). time, large is fitted from 30 buses upward, where arithmetic rather than interpreter overhead dominates.

Solver	Time, all	Time, large	Per-iter., large	Iterations
SNR	0.730	1.470	1.410	0.041
PNR	0.920	1.827	1.369	0.251
PNR+	0.825	1.649	1.458	0.101
RCI	0.827	1.589	1.380	0.141

Table 15: Jacobian derivative speed-up versus system size.

n	Deriv. SNR (μ s)	Deriv. PNR (μ s)	Deriv. ratio	Iter. ratio
3	13.04	11.50	1.134	0.993
5	15.50	11.85	1.308	1.122
6	14.12	11.38	1.240	1.078
14	16.50	12.96	1.273	1.075
24	19.62	14.87	1.319	1.102
30	21.03	16.33	1.288	1.057
57	36.08	36.97	0.976	0.790
118	135.1	89.63	1.507	1.073
300	664.2	416.2	1.596	1.106

Figures 16 and 17 summarise the table; Figures 18 and 19 show convergence *character*. The latter are the most informative plots

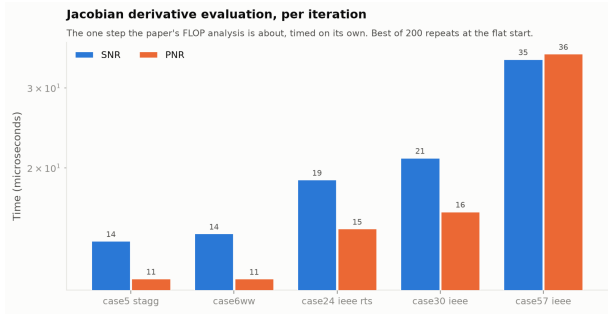


Figure 11: Jacobian derivative evaluation per iteration, timed on its own - the one step the paper’s FLOP analysis is about.

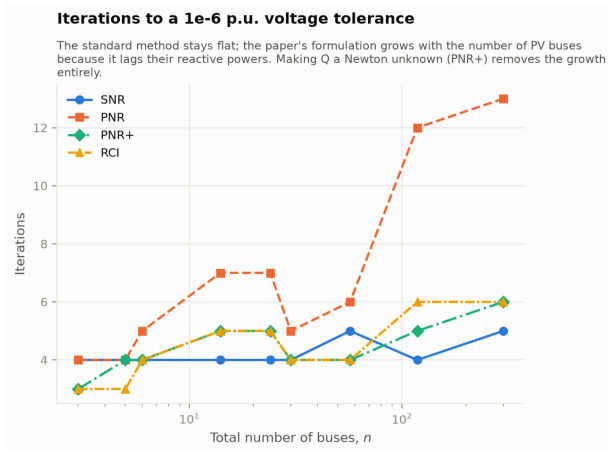


Figure 12: Iterations versus system size – the project’s main finding. The standard method stays flat; the paper’s formulation grows as PV buses multiply.

Table 16: Dense versus sparse storage. speed-up is dense time divided by sparse time; values below 1 mean sparse was slower.

n	Solver	Dense (ms)	Sparse (ms)	Speed-up	Dense J (kB)	Sparse J (kB)
57	SNR	1.76	8.27	0.21	87.8	5.6
57	PNR	2.35	8.52	0.28	87.8	5.6
118	SNR	2.97	6.93	0.43	255.9	8.2
118	PNR	8.49	18.11	0.47	255.9	8.2
300	SNR	22.27	11.39	1.96	2 195	29.2
300	PNR	55.57	26.92	2.06	2 195	29.2

in the study: on a semi-logarithmic scale a straight line is linear convergence and a curve that steepens is quadratic. SNR, PNR+ and RCI curve; FDLF and GS are straight; and PNR, which should curve, is visibly straight on the 118-bus system – direct evidence that the lagged- Q treatment has cost the method its Newton character, not merely a few extra iterations.

All methods agree on the answer. Table 18 reports each solver’s worst-case distance from a standard-NR reference driven to a 10^{-13} p.u.

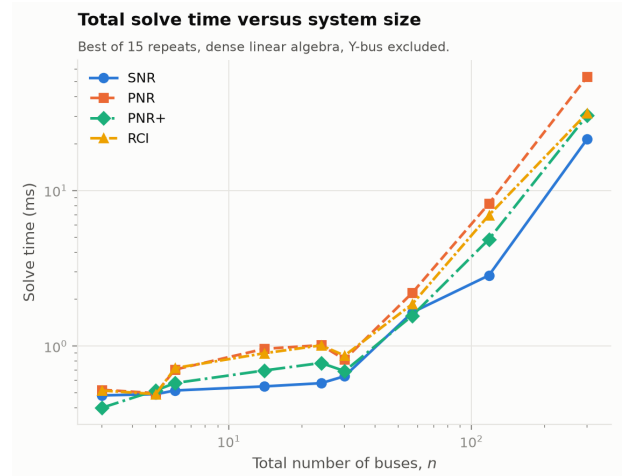


Figure 13: Total solve time versus system size.

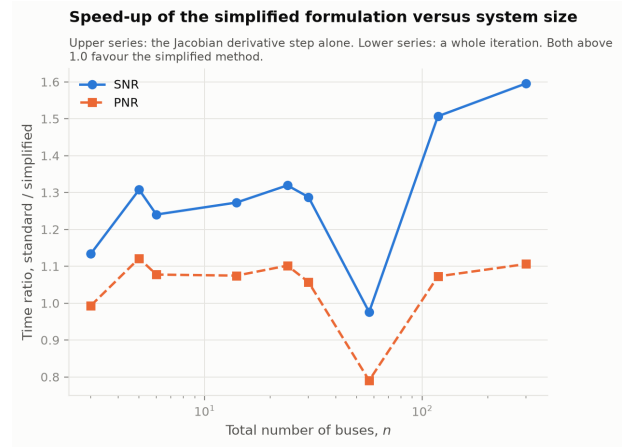


Figure 14: Speed-up of the simplified formulation versus system size. Upper series: the Jacobian derivative step alone. Lower series: a whole iteration.

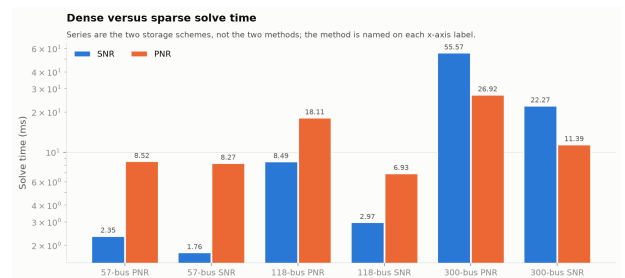
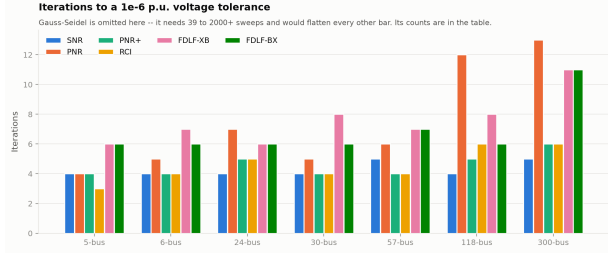
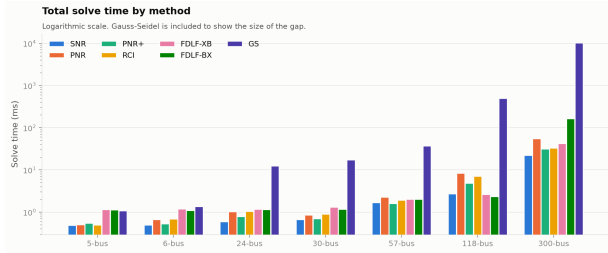


Figure 15: Dense versus sparse solve time. The series are storage schemes, not methods; the crossover sits between 118 and 300 buses.

power mismatch. Every converged run lands inside the tolerance it

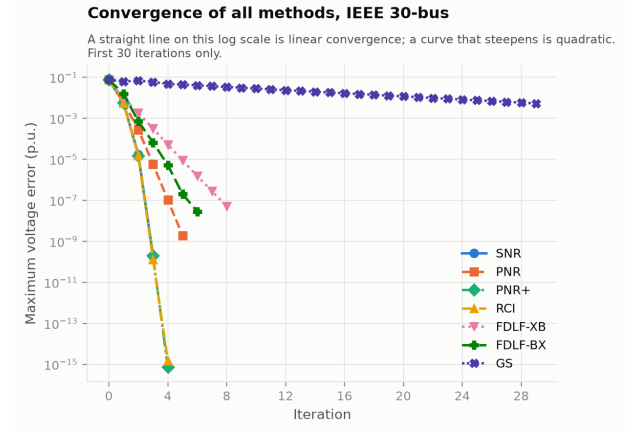
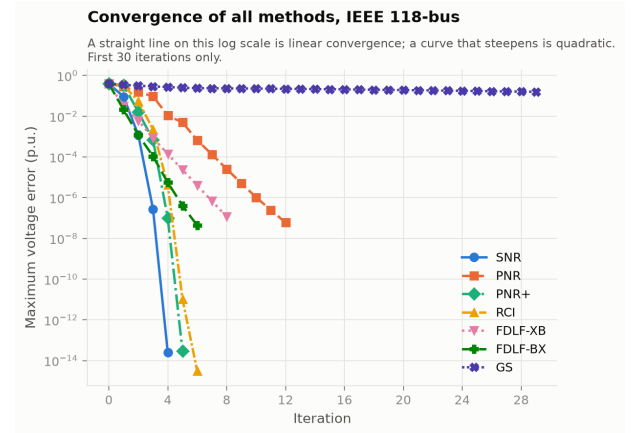
Table 17: All seven solvers on all test systems. Flat start, 10^{-6} p.u. maximum voltage correction; Gauss–Seidel is given a 3000-sweep budget and the Newton methods 60. Best of 10 repeats. – marks non-convergence.

System	n	Iterations							Solve time (ms)						
		SNR	PNR	PNR+	RCI	F-XB	F-BX	GS	SNR	PNR	PNR+	RCI	F-XB	F-BX	GS
case5_stagg	5	4	4	4	3	6	6	27	0.49	0.50	0.55	0.49	1.16	1.14	1.08
case6ww	6	4	5	4	4	7	6	25	0.50	0.67	0.53	0.69	1.19	1.10	1.36
case24_ieee_rts	24	4	7	5	5	6	6	76	0.60	1.03	0.79	1.04	1.17	1.16	12.5
case30_ieee	30	4	5	4	4	8	6	100	0.67	0.86	0.70	0.90	1.32	1.17	17.5
case57_ieee	57	5	6	4	4	7	7	117	1.70	2.27	1.61	1.93	2.03	2.03	37.0
case118_ieee	118	4	12	5	6	8	6	378	2.72	8.42	4.84	7.09	2.63	2.35	502
case300_ieee	300	5	13	6	6	11	11	–	22.4	55.6	31.7	33.1	42.2	165	>10 200

**Figure 16: Iterations by method. Gauss–Seidel is omitted; it needs 25 to 3000+ sweeps and would flatten the scale.****Figure 17: Total solve time by method, logarithmic scale. Gauss–Seidel is included to show the size of the gap.****Table 18: Worst-case distance from a tightly converged reference solution, over all seven test systems.**

Solver	Max. $ V - V_{ref} $ (p.u.)	Note
SNR	2.4e–13	reference formulation
PNR	3.9e–07	within requested 10^{-6}
PNR+	6.5e–14	
RCI	5.5e–14	
FDLF-XB	1.2e–07	
FDLF-BX	1.1e–07	
GS	4.5e–04	non-convergent on EX2

was asked for, which is what makes the timing comparison meaningful at all. The single exception is Gauss–Seidel on the 300-bus system, which did not converge.

**Figure 18: Convergence character on the IEEE 30-bus system.****Figure 19: Convergence character on the IEEE 118-bus system. PNR's trace is a straight line — linear, not quadratic, convergence.**

4.8 Extension 3: Memory

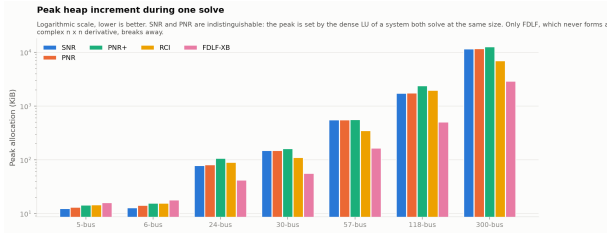
A natural question is whether the simplified formulation also saves memory. Table 19 answers it: **no**.

Table 19: Peak heap increment per solve (kB) and the dimension of the linear system factorised each iteration.

n	Peak memory (kB)					Rows
	SNR	PNR	PNR+	RCI	F-XB	SNR=PNR
5	12.4	13.2	14.5	14.6	16.0	8
6	12.9	14.3	15.7	15.7	18.1	8
24	78.6	81.1	107.2	90.3	42.1	36
30	148.9	150.3	161.8	110.3	56.1	53
57	556.9	559.1	564.1	350.9	165.3	106
118	1 738	1 757	2 398	1 978	506.4	181
300	11 650	11 700	12 680	7 037	2 919	530

Table 20: Dense versus sparse storage of Y_{bus} and the Jacobian (kB).

n	nnz	Density	Y dense	Y sparse	J dense	J sparse
5	19	0.760	0.39	0.39	0.50	0.30
6	28	0.778	0.56	0.57	0.50	0.44
24	92	0.160	9.0	1.90	10.1	1.44
30	112	0.124	14.1	2.31	22.0	1.75
57	213	0.066	50.8	4.39	87.8	3.33
118	476	0.034	217.6	9.76	255.9	7.44
300	1 118	0.012	1 406	23.0	2 195	17.5

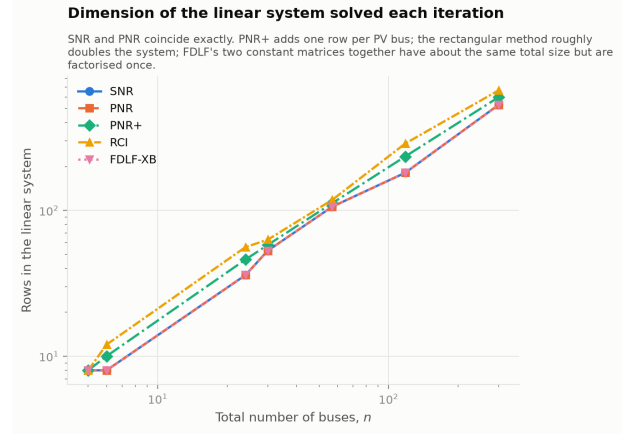
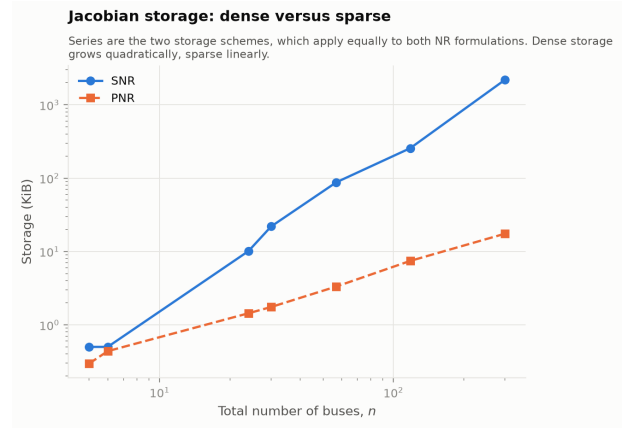
**Figure 20: Peak heap increment during one solve, logarithmic scale. SNR and PNR are indistinguishable.**

SNR and PNR are indistinguishable at every size — within 1.5% at 118 and 300 buses. The reason is structural and is visible in the last column: the reformulation changes the Jacobian’s *contents*, not its *dimension*, and the peak allocation is set by the dense LU factorisation of that same-sized system. PNR+ and RCI pay for their extra unknowns with larger systems (234 and 287 rows against 181 at 118 buses). FDLF is in a different class entirely — 506 kB against 1 738 kB at 118 buses — because it never forms a complex $n \times n$ derivative matrix at all.

The paper’s contribution is therefore a time optimisation, not a space one. The space lever is sparsity, and Table 20 shows how large it is: at 300 buses, dense storage of Y_{bus} and the Jacobian costs 126× more than sparse storage. Figures 20–22 present the three views.

4.9 Extension 4: Robustness Under Load

Nominal-case performance is not the whole engineering question. Every injection was scaled by a loading factor from 1.0 to 5.0 in

**Figure 21: Dimension of the linear system solved each iteration. SNR and PNR coincide exactly; PNR+ adds one row per PV bus.****Figure 22: Jacobian storage, dense versus sparse. The series are storage schemes, which apply equally to both NR formulations.****Table 21: Largest loading factor solved from a flat start. “censored” means the method never failed inside the sweep, so its true limit is higher.**

System	SNR	PNR	PNR+	RCI	FDLF-XB
case30_ieee	≥ 5.0	≥ 5.0	≥ 5.0	≥ 5.0	≥ 5.0
case57_ieee	1.8	1.8	1.8	1.8	1.8
case118_ieee	3.0	1.6	2.0	3.0	3.0

steps of 0.2 and each method re-solved from a flat start. Table 21 reports the largest factor each method still solves.

The three systems separate the two possible explanations cleanly. On the 30-bus case no method fails anywhere in the sweep. On the 57-bus case all five fail at exactly the same factor, 2.0 — a common ceiling reached simultaneously is a *physical* limit, the network’s voltage-stability (nose) point, and no solver can pass it. On the

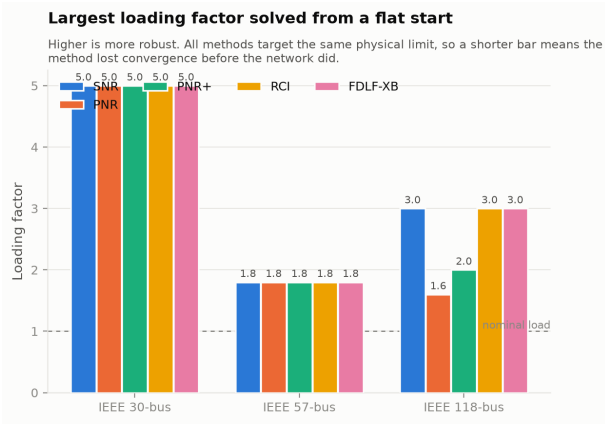


Figure 23: Largest loading factor solved from a flat start. Higher is more robust.

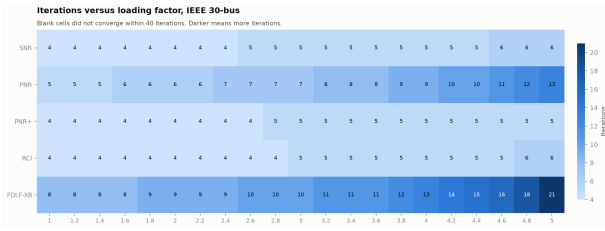


Figure 24: Iteration count against loading factor, IEEE 30-bus. No method fails inside the sweep.

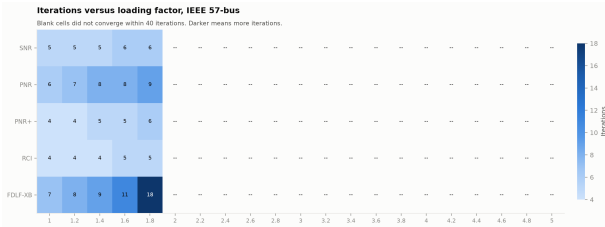


Figure 25: IEEE 57-bus. All methods stop at the same loading factor — a physical voltage-stability limit.

118-bus case the methods separate, which means the failures there are *numerical* rather than physical: SNR, RCI and FDLF all reach 3.0, PNR+ reaches 2.0, and PNR gives up at 1.6 — barely past nominal loading.

This is the most practically significant finding of the project. The base paper’s formulation is not merely slower on large PV-heavy systems; it has a materially smaller region of convergence, forfeiting almost half the usable loading range on the 118-bus system relative to the standard method. That PNR+ recovers part of the gap, and RCI — also a current-mismatch method — loses none of it, confirms once more that the cause is the lagged- Q approximation rather than current-mismatch formulations as such. Figure 23 shows the limits and Figures 24–26 give the full iteration surface.

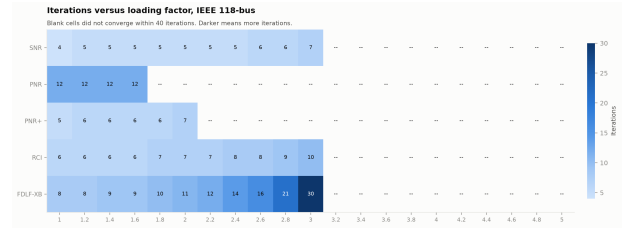


Figure 26: IEEE 118-bus. The methods separate, so the failures are numerical: PNR (blank from 1.8 onward) loses convergence first.

Table 22: One Newton iteration timed stage by stage (μ s), and the cost of a chord iteration, which pays only for the residual and the substitution. ratio is Newton iteration cost divided by chord iteration cost. Timings are single-threaded (Section 3.3).

n	Resid.	J build (share)	LU fact. (share)	Subst.	Newton iter.	Chord iter. (ratio)
3	21.8	51.3 (60%)	4.7 (6%)	7.1	84.9	28.9 (2.9 $\times$)
5	17.2	52.3 (64%)	5.3 (6%)	7.5	82.2	24.7 (3.3 $\times$)
6	22.4	53.4 (60%)	5.3 (6%)	7.5	88.7	29.9 (3.0 $\times$)
24	22.8	61.6 (59%)	11.7 (11%)	7.9	104.0	30.7 (3.4 $\times$)
30	23.5	69.3 (57%)	19.7 (16%)	8.8	121.2	32.2 (3.8 $\times$)
57	25.3	111.8 (53%)	64.9 (31%)	10.6	212.7	35.9 (5.9 $\times$)
118	36.3	222.5 (47%)	196.3 (42%)	15.6	470.6	51.8 (9.1 $\times$)
300	75.6	1353.2 (30%)	2961.4 (67%)	56.8	4446.9	132.3 (33.6 $\times$)

4.10 Extension 5: Amortising the Factorisation

Section 4.5 showed that the majority of a Newton iteration is work the base paper’s reformulation cannot touch. That observation suggests a different attack on the same cost: instead of making the Jacobian *cheaper to build*, do not rebuild it at all. The *chord* variants build and LU-factorise the Jacobian once at the flat start and then reuse the stored factors through forward and back substitution for every subsequent step. Both formulations are treated this way, giving SNR-chord and PNR-chord.

The cost profile inverts with system size. Table 22 times each stage of a single iteration in isolation. The result reframes the whole study. On the small systems the paper works with, building the Jacobian really is the dominant cost — 60–64% of an iteration at 5 and 6 buses, against 6% for the factorisation. By 300 buses the proportions have reversed: the build is 30% and the LU factorisation is 67%. The base paper therefore optimises precisely the stage whose relative importance *shrinks* as systems grow, and leaves untouched the one that comes to dominate. Reducing a 30% stage by the audited factor of 1.6 removes about 11% of an iteration; eliminating the 67% stage removes 67%.

A chord iteration costs 2.9 $\times$ less than a Newton iteration at 3 buses and 33.6 $\times$ less at 300, because it pays only for the residual and a pair of triangular solves. Figure 27 plots the breakdown.

The trade is favourable, and increasingly so. Freezing the Jacobian costs the method its quadratic convergence — a chord iteration is a stationary linear iteration — so more steps are needed. Table 23 shows the balance. SNR-chord needs 16 iterations at 300 buses

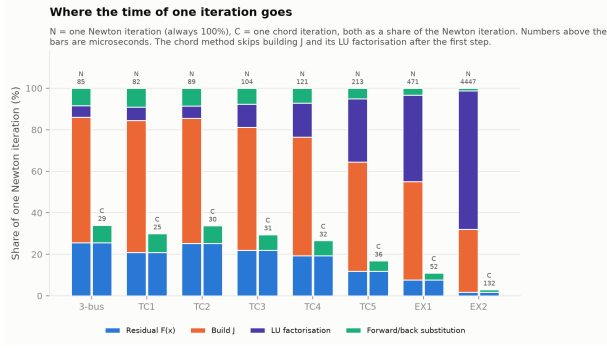


Figure 27: Cost of each stage of an iteration against system size. The factorisation overtakes the Jacobian build between 118 and 300 buses.

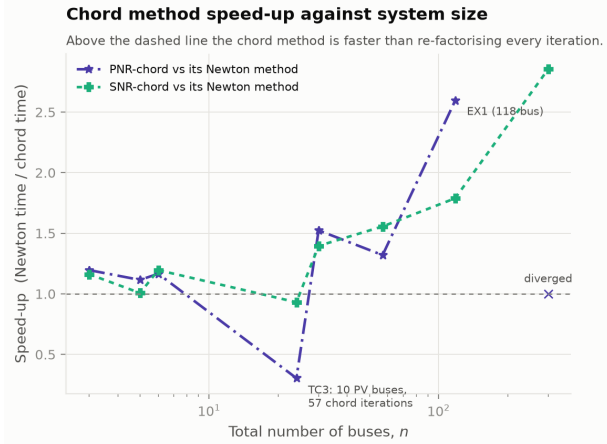


Figure 28: Chord speed-up against system size. Values above 1 mean freezing the Jacobian was worth the extra iterations.

against standard NR’s 5, but each is $33.6\times$ cheaper, and the net result is a $2.85\times$ end-to-end speed-up. That is the largest whole-solve gain measured anywhere in this project: larger than the $1.96\times$ available from sparse storage (Table 16), and far larger than the $\approx 1.0\times$ delivered by the base paper’s reformulation (Table 11). It is also obtained with a *single* LU factorisation instead of the 5–13 the Newton methods perform.

The speed-up is not uniform — it is roughly break-even on the smallest systems, where a single iteration is dominated by interpreter overhead rather than arithmetic — but it rises monotonically with size from 30 buses upward ($1.40, 1.56, 1.79, 2.85$), which is the opposite of the trend for the paper’s reformulation. Figure 28 plots it, and Figures 29 and 30 give the underlying iteration counts and times.

The strategy works on the standard formulation, not on the paper’s. SNR-chord converges on every system tested. PNR-chord does not: it fails outright on the 300-bus case, and on the 24-bus system — the most PV-dense in the set, with 10 of 24 buses voltage-controlled — its linear convergence rate degrades to 0.785, requiring 57 iterations

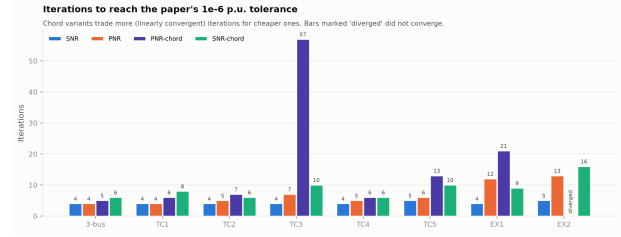


Figure 29: Iterations required by the Newton and chord variants of both formulations.

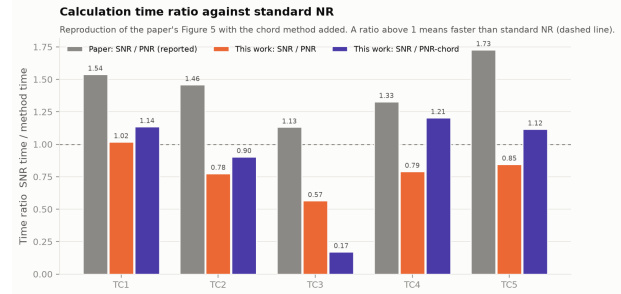


Figure 30: Solve time of all four variants on the paper’s test cases and the two extension systems.

and making it $3.3\times$ slower than ordinary PNR. The explanation is the one that has recurred throughout this report. Freezing the Jacobian already reduces the method to a linear iteration; the paper’s lagged- Q treatment is a second linear iteration layered on top, and the two contraction factors compound. Measured chord convergence rates make this explicit, and tie the degradation to the PV-bus fraction rather than to size. On the three PV-dense systems — 24-, 118- and 300-bus, with 42%, 45% and 23% of buses voltage-controlled — PNR-chord contracts at 0.79, 0.57 and 0.61 against 0.24–0.40 for SNR-chord. On the 57-bus system, where only 11% of buses are PV, it recovers to 0.34.

Figure 31 shows the convergence traces. The chord curves are straight lines, as a stationary linear iteration must be, and their slopes are the rates quoted above; the per-iteration data is tabulated in `08_convergence_histories`.

What this says about the base paper. The paper’s premise — that the Jacobian rebuild is the cost worth attacking — is correct, and this experiment confirms it from the opposite direction. Its chosen remedy is simply far too conservative. Making the rebuild $1.6\times$ cheaper saves about 11% of an iteration at 300 buses; skipping the rebuild and the factorisation together saves 97% of one. The strategy the paper points toward is sound; carried to its conclusion it yields $2.85\times$ rather than $1.0\times$, and it does so on the standard formulation that the paper set out to improve upon.

4.11 Threats to Validity

Three limitations should be stated. First, timings come from a reference implementation in Python and NumPy; absolute numbers would differ in a compiled solver, though the *ratios* — which is

Table 23: Chord iterations against their Newton counterparts. – marks non-convergence within the 150-iteration budget. Absolute times are not comparable with Tables 11 and 17: this experiment pins BLAS to a single thread so that substitution-only iterations are timed fairly. Within-table ratios are unaffected.

System	n	n_{pv}	Iterations				Solve time (ms)				Chord speed-up	
			SNR	PNR	PNR-ch	SNR-ch	SNR	PNR	PNR-ch	SNR-ch	vs. PNR	vs. SNR
case3_saadat	3	1	4	4	5	6	0.50	0.53	0.44	0.43	1.20	1.16
case5_stagg	5	0	4	4	6	8	0.50	0.49	0.44	0.50	1.11	1.01
case6ww	6	2	4	5	7	6	0.49	0.63	0.54	0.41	1.17	1.20
case24_ieee_rts	24	10	4	7	57	10	0.57	1.01	3.32	0.61	0.30	0.93
case30_ieee	30	5	4	5	6	6	0.65	0.82	0.54	0.47	1.52	1.40
case57_ieee	57	6	5	6	13	10	1.24	1.47	1.11	0.80	1.32	1.56
case118_ieee	118	53	4	12	21	9	2.07	5.94	2.29	1.16	2.59	1.79
case300_ieee	300	68	5	13	–	16	27.93	62.65	–	9.79	–	2.85

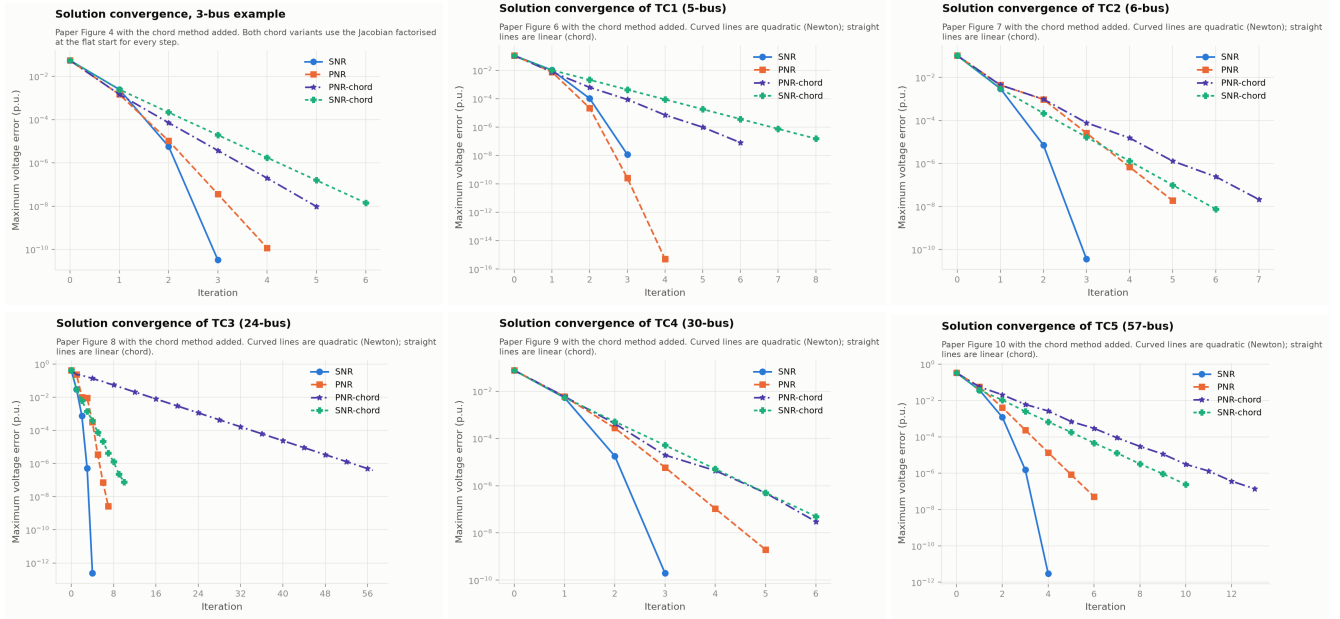


Figure 31: Convergence of the Newton and chord variants. Top row, left to right: 3-bus worked example, TC1 (5-bus), TC2 (6-bus). Bottom row: TC3 (24-bus), TC4 (30-bus), TC5 (57-bus). The chord traces are straight - linear convergence — and PNR-chord on TC3 is nearly flat.

what every claim here rests on — are far more stable, and the stage decomposition of Table 12 isolates the arithmetic from interpreter overhead. Second, the paper’s test systems were modified in an unstated way, so its Table 3 and its exact iteration counts are not strictly comparable with ours; the comparison is made on the unmodified library cases and this is flagged wherever it matters. Third, the 57-bus derivative-ratio outlier noted in Section 4.5 is reported as measured and not explained away, and the 57-bus loading limit is a property of that network rather than of any solver.

5 Conclusion and Future Work

5.1 Key Takeaways

This project set out to determine why the base paper’s predicted speed-up of roughly $n/2$ appears in its own experiments as a factor

of 1.13–1.73. The answer has three parts, and each is supported by an independent line of evidence.

1. *The implementation is faithful.* Everything the paper prints and that can be checked, checks out: the Y-bus to four decimals, the first Jacobian to every printed digit, the mismatch and correction vectors through three iterations, and the converged solution of Table 2 (Section 4.1). The disagreements reported later are therefore disagreements with the paper’s analysis, not artefacts of a different model.

2. *The paper’s complexity analysis is wrong, but its conclusion is right.* Table 1 of the paper counts off-diagonal Jacobian work once per matrix row rather than once per entry, which is what makes the cost appear linear in n . A dense Jacobian cannot be filled in linear time, because each of its $O(n^2)$ entries carries a distinct Y_{ki} . An element-by-element recount gives both methods a quadratic cost

separated by a constant factor of 1.67–1.80. That is the size of saving the paper’s own Table 5 reports (1.13–1.73 $\times$); its Table 1 would have predicted 25.9 $\times$ at 57 buses. The corrected model is the right order of magnitude throughout; the published model overestimates by a factor of fifteen (Section 4.2).

3. *The saving is real but structurally confined, and is more than cancelled by a flaw in the PV handling.* Three independent measurements converge on this. The derivative stage alone speeds up by 1.21–1.31 $\times$ on the paper’s own test systems and 1.51–1.60 $\times$ at 118 and 300 buses, rising with system size as predicted, with a single outlier at 57 buses measuring 0.98 $\times$ (Tables 12 and 15). But that stage is only a quarter of an iteration; 51–74% of the work is block assembly and LU factorisation, which are identical in the two formulations, so the whole-iteration ratio is unity to within noise. And the paper’s treatment of PV-bus reactive power — re-estimating Q from the current voltage guess instead of solving for it — layers a successive-substitution fixed point on top of Newton’s method and costs it its quadratic convergence. The effect is small on the paper’s own test systems and severe beyond them: 13 iterations against 5 at 300 buses, a fitted iteration-growth exponent six times that of the standard method, convergence traces that are visibly straight rather than curved on a semi-log plot, and a loss of almost half the usable loading range on the 118-bus system (Sections 4.6–4.9).

4. *The defect is fixable.* PNR+, which makes Q a genuine Newton unknown at PV buses, restores an almost-flat iteration count (exponent 0.101 against PNR’s 0.251), stays within one iteration of standard NR on every system tested, and is 1.77 $\times$ faster than the published method at 300 buses. RCI, a different current-mismatch formulation that never lags Q , shows the same healthy behaviour. The deficiency therefore belongs to one specific approximation, not to current-mismatch power flow in general.

5. *The paper’s premise is right; its remedy is too conservative.* Freezing the Jacobian and reusing its LU factors — the limit of the strategy the base paper pursues — gives a 2.85 $\times$ end-to-end speed-up at 300 buses, the largest gain measured in this study, using a single factorisation instead of five. Stage timing explains why: at that size the factorisation is 67% of an iteration and the Jacobian build the paper optimises only 30%, so the published remedy can recover at most about 11% of an iteration while the chord strategy recovers 97%. That the chord variant works on the *standard* formulation but diverges on the paper’s — whose lagged- Q fixed point compounds with the frozen Jacobian’s own linear rate — is the final confirmation that the PV treatment, not the current-mismatch idea, is what limits the method (Section 4.10).

Practical recommendation. On a dense implementation of a small, well-conditioned, PV-light network, the paper’s method is a modest and genuine improvement. On anything that resembles a modern application — sparse storage, hundreds of buses, many voltage-controlled generators, stressed operating points — it should be adopted only with the augmented PV handling, and even then the larger levers are sparse linear algebra (a 2 $\times$ gain at 300 buses and growing, Table 16) and amortising the factorisation — whether through FDLF-BX, the fastest solver at 118 buses (Table 17), or through chord iterations, the fastest at 300 (Table 23).

5.2 Limitations

The study is confined to balanced, positive-sequence, single-phase equivalent transmission networks up to 300 buses, solved from a flat start with a reference Python implementation. Generator reactive limits are implemented but disabled in the benchmarks, so that every method solves exactly the same problem. Timings reflect NumPy/SciPy performance rather than that of a compiled production solver, and the dense/sparse crossover in particular is a property of SciPy’s sparse LU and would move in another library.

5.3 Future Work

Four directions follow naturally from the results above.

- (1) **Adaptive Jacobian refresh.** The chord study of Section 4.10 refreshes the factorisation never; Newton refreshes it every step. Neither is optimal. A policy that refreshes when the observed contraction rate deteriorates past a threshold should dominate both, and would also rescue PNR-chord on the PV-dense cases where its rate degrades to 0.785. The solver already accepts a `refresh_every` parameter, so the experiment is a sweep away.
- (2) **Sparse-native Jacobian assembly.** The present implementation assembles dense derivative matrices and converts. Building directly on the Y_{bus} sparsity pattern would move the dense/sparse crossover well below 300 buses and let the audited sparse model of Table 9 be validated against the clock.
- (3) **Larger and more stressed systems.** Extending to 1354-, 2869- and 9241-bus European cases, and to continuation power flow near the nose point, would test whether PNR’s loss of robustness worsens as predicted.
- (4) **A formal convergence analysis of the lagged- Q iteration.** This report establishes the loss of quadratic convergence empirically. Deriving the contraction factor of the outer fixed point as a function of the PV-bus count and network strength would turn an observation into a prediction, and would say in advance which networks the published method is safe on.

5.4 Reproducibility

Every number, table and figure in this report is regenerated from source. The implementation, the test systems and the complete result set are publicly available at

https://github.com/Diganta81/numerical_project_402

The main result set regenerates with `python run_all.py` in approximately two minutes; the chord study of Section 4.10 is a separate script, `scripts/08_ext_chord_method.py`; and the 285-test suite runs with `python -m pytest`. Every figure has a companion table of the same name under `results/`, written as CSV and Markdown, carrying the numbers behind it, so each claim made above can be checked against the data that produced it without rerunning anything.

References

- [1] T. Kulworawanichpong. 2010. Simplified Newton–Raphson power-flow solution method. *International Journal of Electrical Power & Energy Systems* 32, 6 (2010), 551–558.

- [2] W. F. Tinney and C. E. Hart. 1967. Power Flow Solution by Newton's Method. *IEEE Transactions on Power Apparatus and Systems* PAS-86, 11 (1967), 1449–1460.
- [3] B. Stott and O. Alsac. 1974. Fast Decoupled Load Flow. *IEEE Transactions on Power Apparatus and Systems* PAS-93, 3 (1974), 859–869.
- [4] V. M. da Costa, N. Martins, and J. L. R. Pereira. 1999. Developments in the Newton Raphson Power Flow Formulation Based on Current Injections. *IEEE Transactions on Power Systems* 14, 4 (1999), 1320–1326.
- [5] G. W. Stagg and A. H. El-Abiad. 1968. *Computer Methods in Power System Analysis*. McGraw-Hill, New York.
- [6] H. Saadat. 1999. *Power System Analysis*. McGraw-Hill, New York.
- [7] A. J. Wood, B. F. Wollenberg, and G. B. Sheblé. 2014. *Power Generation, Operation, and Control* (3rd ed.). Wiley, Hoboken, NJ.
- [8] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas. 2011. MATPOWER: Steady-State Operations, Planning, and Analysis Tools for Power Systems Research and Education. *IEEE Transactions on Power Systems* 26, 1 (2011), 12–19.
- [9] University of Washington. 1999. Power Systems Test Case Archive. <https://labs.ece.uw.edu/pstca/>
- [10] C. R. Harris et al. 2020. Array programming with NumPy. *Nature* 585 (2020), 357–362.
- [11] P. Virtanen et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17 (2020), 261–272.
- [12] J. D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95.
- [13] W. McKinney. 2010. Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference*. 56–61.